

DEPARTAMENTO
DE COMPUTACION

Facultad de Ciencias Exactas y Naturales - UBA

Sistemas Digitales

La abstracción de la información

Sistemas Digitales

2do Cuatrimestre 2026

FCEN - UBA

Índice

- Álgebra de Boole
- Abstracción digital
- Compuertas, tablas de verdad y clasificación de sistemas
- Representación de la información - Introducción: enteros sin signo

Álgebra de Boole

George Boole

George Boole, matemático inglés (1815-1864).

Escribe *The Laws of Thought* (1854), donde desarrolla un formalismo matemático para razonar sobre proposiciones lógicas.

En esta obra, Boole demuestra que la lógica puede representarse mediante un álgebra, y que las proposiciones pueden representarse mediante variables binarias (0 y 1).



Claude Shannon

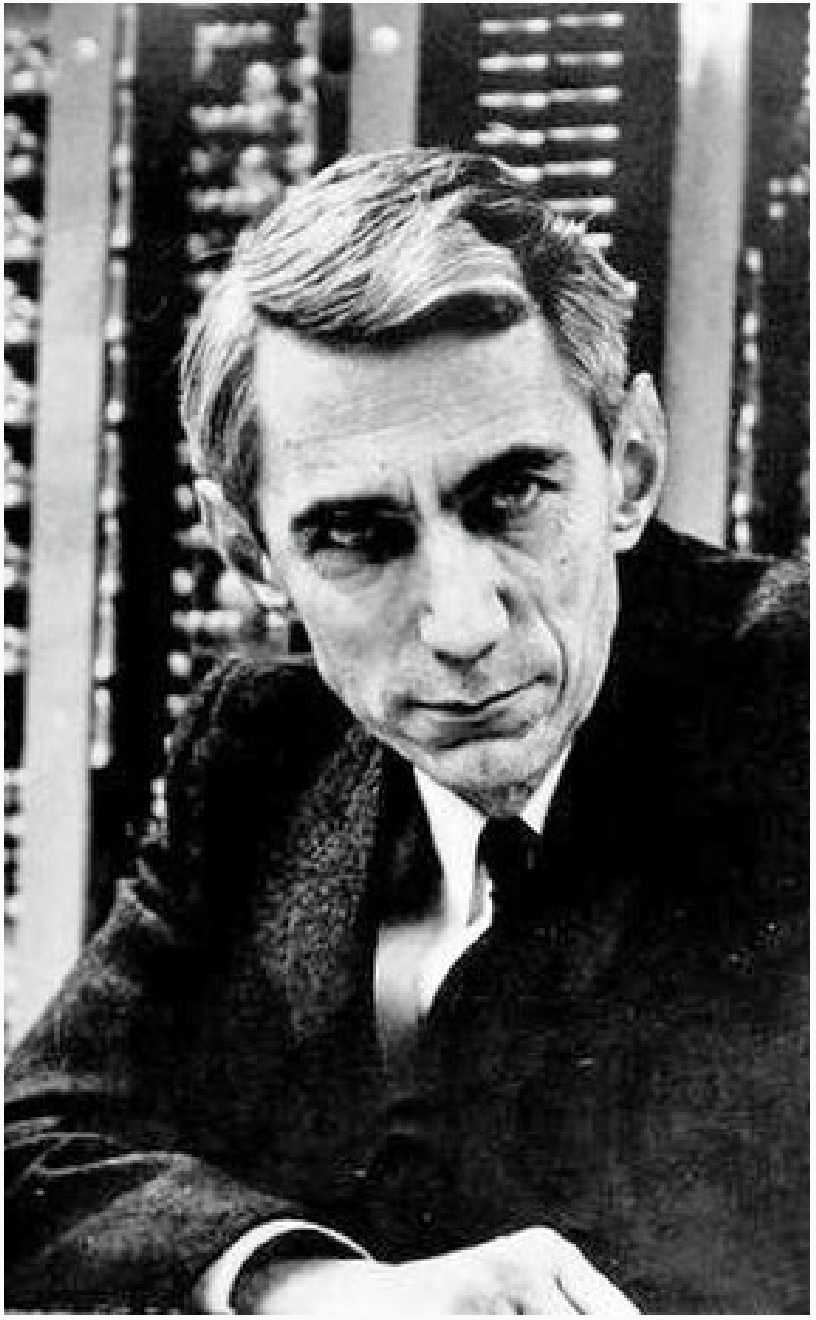
Claude Shannon, matemático, ingeniero eléctrico y criptógrafo (1916-2001).

Escribe ***A Symbolic Analysis of Relay and Switching Circuits*** (1937), donde demuestra que el formalismo de Boole puede aplicarse al diseño de sistemas digitales.

En 1948, publica ***A Mathematical Theory of Communication***, donde desarrolla la teoría de la información, que formaliza el concepto de **bit** como unidad de información.

Por sus numerosas contribuciones, Shannon es considerado el *padre de la criptografía moderna y de la teoría de la información*.

Es reconocido como uno de los científicos más influyentes del siglo XX.,



Abstracción Digital

A partir de un conjunto de **axiomas**, podemos diseñar sistemas digitales arbitrariamente complejos, y razonar sobre ellos sin necesidad de conocer los detalles de su implementación.

Axiomas

Definimos un álgebra de boole como un conjunto B con dos elementos, 0 y 1 donde se definen dos operaciones básicas ($+$ y $\cdot$) que cumplen los siguientes axiomas:

#	Descripción	Axioma	Dual
A1	Existen dos elementos	$B = \{0, 1\}$	

Axiomas

Definimos un álgebra de boole como un conjunto B con dos elementos, 0 y 1 donde se definen dos operaciones básicas ($+$ y $\cdot$) que cumplen los siguientes axiomas:

#	Descripción	Axioma	Dual
A1	Existen dos elementos	$B = \{0, 1\}$	
A2	operador negación:	$\overline{0} = 1$	$\overline{1} = 0$

Axiomas

Definimos un álgebra de boole como un conjunto B con dos elementos, 0 y 1 donde se definen dos operaciones básicas ($+$ y $\cdot$) que cumplen los siguientes axiomas:

#	Descripción	Axioma	Dual
A1	Existen dos elementos	$B = \{0, 1\}$	
A2	operador negación:	$\overline{0} = 1$	$\overline{1} = 0$
A3	*idempotencia*	$0 \cdot 0 = 0$	$1 + 1 = 1$

Axiomas

Definimos un álgebra de boole como un conjunto B con dos elementos, 0 y 1 donde se definen dos operaciones básicas ($+$ y $\cdot$) que cumplen los siguientes axiomas:

#	Descripción	Axioma	Dual
A1	Existen dos elementos	$B = \{0, 1\}$	
A2	operador negación:	$\overline{0} = 1$	$\overline{1} = 0$
A3	*idempotencia*	$0 \cdot 0 = 0$	$1 + 1 = 1$
A4		$1 \cdot 1 = 1$	$0 + 0 = 0$

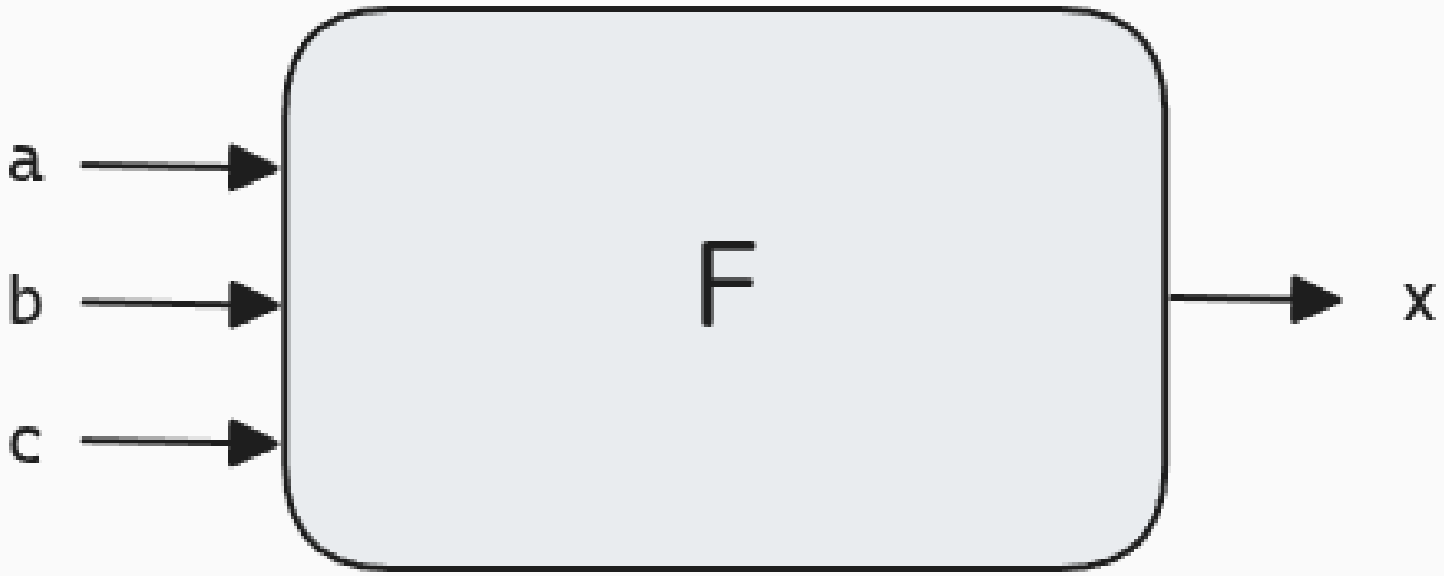
Axiomas

Definimos un álgebra de boole como un conjunto B con dos elementos, 0 y 1 donde se definen dos operaciones básicas ($+$ y $\cdot$) que cumplen los siguientes axiomas:

#	Descripción	Axioma	Dual
A1	Existen dos elementos	$B = \{0, 1\}$	
A2	operador negación:	$\overline{0} = 1$	$\overline{1} = 0$
A3	*idempotencia*	$0 \cdot 0 = 0$	$1 + 1 = 1$
A4		$1 \cdot 1 = 1$	$0 + 0 = 0$
A5	elemento neutro	$0 \cdot 1 = 1 \cdot 0 = 0$	$0 + 1 = 1 + 0 = 1$

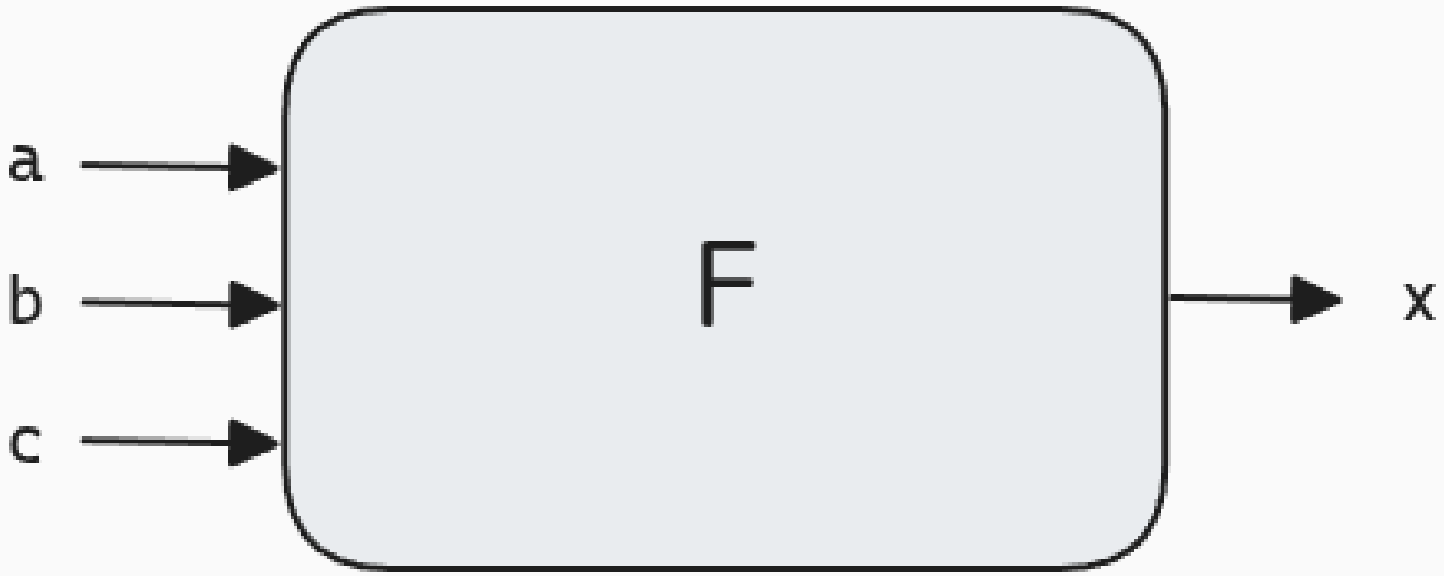
De un problema a una función

Un sistema digital puede pensarse como una **caja negra** que transforma entradas en salidas:



De un problema a una función

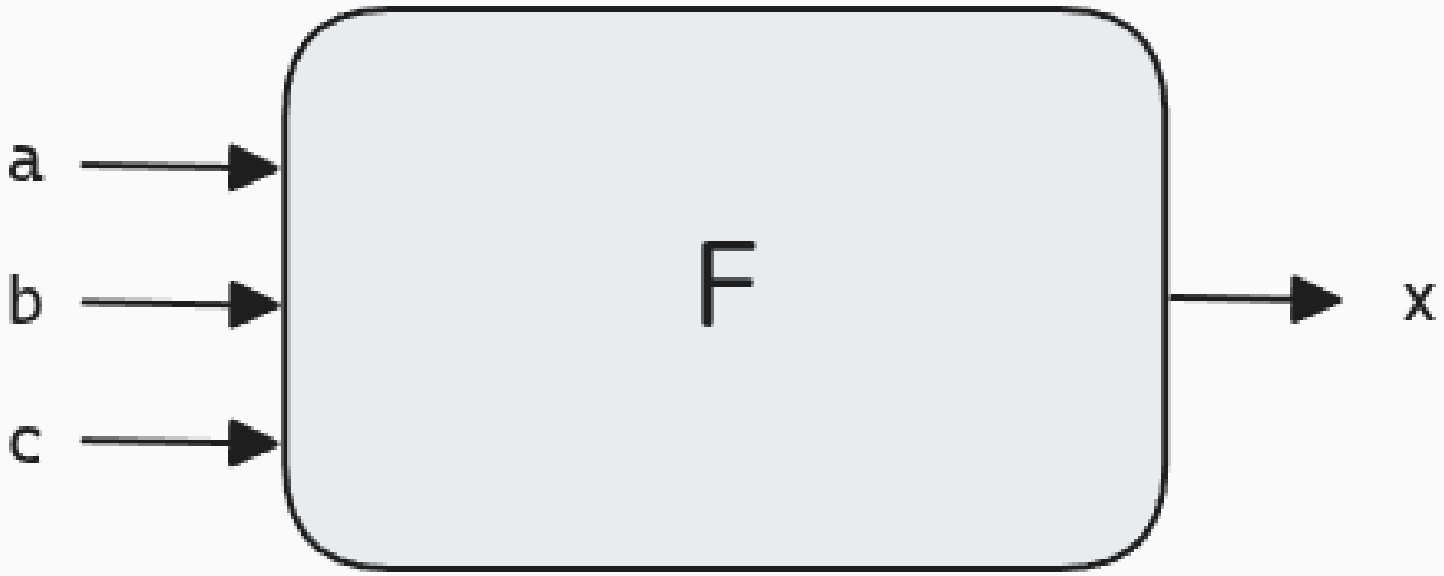
Un sistema digital puede pensarse como una **caja negra** que transforma entradas en salidas:



No hace falta conocer todavía **cómo** está implementado el sistema: alcanza con describir **qué hace**.

De un problema a una función

Un sistema digital puede pensarse como una **caja negra** que transforma entradas en salidas:



No hace falta conocer todavía **cómo** está implementado el sistema: alcanza con describir **qué hace**.

Necesitamos una forma de describir la relación entre entradas y salidas: eso es lo que llamamos una **función booleana**.

Función booleana

Para el sistema de arriba, podemos definir

$$x = F(a, b, c)$$

donde F es una función booleana que depende de las entradas a, b y c, y produce la salida x.

Por ejemplo, la ecuación booleana

$$x = a + \overline{b} \cdot c$$

describe en forma completa el comportamiento del sistema

Una forma de especificar una función booleana es mediante una **ecuación booleana**, que describe la salida en función de las entradas.

Precedencia de operadores

Formalmente las operaciones booleanas son simétricas:

$$a + (b.c) = (a + b).(a + c)$$ 😮

Precedencia de operadores

Formalmente las operaciones booleanas son simétricas:

$$a + (b.c) = (a + b).(a + c)$$ 😮

Para interpretar una expresión booleana inequívocamente, se adopta universalmente la siguiente precedencia:

$$() > \overline{x} > \cdot > +$$

Precedencia de operadores

Formalmente las operaciones booleanas son simétricas:

$$a + (b.c) = (a + b).(a + c)$$ 😮

Para interpretar una expresión booleana inequívocamente, se adopta universalmente la siguiente precedencia:

$$() > \overline{x} > \cdot > +$$

Entonces, $x = a + \overline{b} \cdot c$ tiene una única interpretación posible:

$$x = a + \left(\overline{b} \cdot c\right)$$

Precedencia de operadores

Formalmente las operaciones booleanas son simétricas:

$$a + (b.c) = (a + b).(a + c)$$ 😮

Para interpretar una expresión booleana inequívocamente, se adopta universalmente la siguiente precedencia:

$$() > \overline{x} > \cdot > +$$

Entonces, $x = a + \overline{b} \cdot c$ tiene una única interpretación posible:

$$x = a + \left(\overline{b} \cdot c\right)$$

Si queremos $x = \left(a + \overline{b}\right) \cdot c$ tenemos que usar sí o sí los paréntesis.

Precedencia de operadores

Formalmente las operaciones booleanas son simétricas:

$$a + (b.c) = (a + b).(a + c)$$ 😮

Para interpretar una expresión booleana inequívocamente, se adopta universalmente la siguiente precedencia:

$$() > \overline{x} > \cdot > +$$

Entonces, $x = a + \overline{b} \cdot c$ tiene una única interpretación posible:

$$x = a + \left(\overline{b} \cdot c\right)$$

Si queremos $x = \left(a + \overline{b}\right) \cdot c$ tenemos que usar sí o sí los paréntesis.

Por último, la notación $\overline{a.b}$ significa $\overline{(a.b)}$.

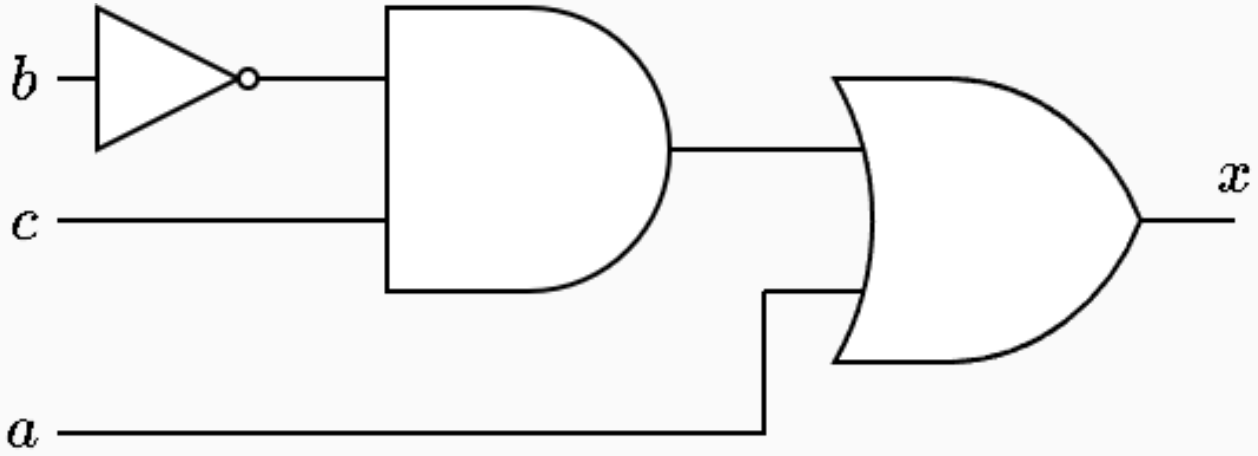
Tabla de verdad

Podemos describir el sistema de arriba mediante una **tabla de verdad**, que muestra todas las combinaciones posibles de entradas y la salida correspondiente:

a	b	c	$x = a + \overline{b} \cdot c$
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Diagrama esquemático

- La misma función booleana $x = a + \overline{b} \cdot c$ se puede ver en el siguiente diagrama de compuertas:



HDL: SystemVerilog

En la materia usaremos un **HDL (Hardware Description Language)** para describir circuitos digitales: **SystemVerilog**. Es un lenguaje de programación que permite describir el comportamiento de sistemas digitales, y que puede ser simulado y sintetizado en hardware.

HDL: SystemVerilog

En la materia usaremos un **HDL (Hardware Description Language)** para describir circuitos digitales: **SystemVerilog**. Es un lenguaje de programación que permite describir el comportamiento de sistemas digitales, y que puede ser simulado y sintetizado en hardware.

La misma función booleana $x = a + \overline{b} \cdot c$ se describe en SystemVerilog de esta manera:

```sv
module ejemplo (
    input  logic a, b, c,
    output logic x
);
    assign x = a | (~b & c);
endmodule
```

Ejemplo: de un problema a una función

Problema: diseñamos un detector de vampiros. La alarma debe sonar si es de noche y la persona no lleva ajo, o si no se refleja en el espejo.

Ejemplo: de un problema a una función

Problema: diseñamos un detector de vampiros. La alarma debe sonar si es de noche y la persona no lleva ajo, o si no se refleja en el espejo.

Identificamos las señales involucradas:

- N: es de noche
- A: la persona lleva ajo
- R: la persona se refleja en el espejo
- V: suena la alarma (¡hay un vampiro!)

Ejemplo: de las palabras a la función

Traducimos cada parte del enunciado a álgebra de Boole:

Ejemplo: de las palabras a la función

Traducimos cada parte del enunciado a álgebra de Boole:

“es de **noche**” $\rightarrow$

Ejemplo: de las palabras a la función

Traducimos cada parte del enunciado a álgebra de Boole:

"es de **noche**" $\rightarrow N$

Ejemplo: de las palabras a la función

Traducimos cada parte del enunciado a álgebra de Boole:

"es de **noche**" $\rightarrow N$

"**no** lleva ajo" $\rightarrow$

Ejemplo: de las palabras a la función

Traducimos cada parte del enunciado a álgebra de Boole:

"es de **noche**" $\rightarrow N$

"**no** lleva ajo" $\rightarrow \overline{A}$

Ejemplo: de las palabras a la función

Traducimos cada parte del enunciado a álgebra de Boole:

"es de **noche**" $\rightarrow N$

"**no** lleva ajo" $\rightarrow \overline{A}$

"...*y*..." $\rightarrow$

Ejemplo: de las palabras a la función

Traducimos cada parte del enunciado a álgebra de Boole:

"es de **noche**" $\rightarrow N$

"**no** lleva ajo" $\rightarrow \overline{A}$

"...*y*..." $\rightarrow$ AND $(\cdot)$

Ejemplo: de las palabras a la función

Traducimos cada parte del enunciado a álgebra de Boole:

"es de **noche**" $\rightarrow N$

"**no** lleva ajo" $\rightarrow \overline{A}$

"...*y*..." $\rightarrow$ AND $(\cdot)$

"**no** se refleja en el espejo" $\rightarrow$

Ejemplo: de las palabras a la función

Traducimos cada parte del enunciado a álgebra de Boole:

"es de **noche**" $\rightarrow N$

"**no** lleva ajo" $\rightarrow \overline{A}$

"...*y*..." $\rightarrow$ AND $(\cdot)$

"**no** se refleja en el espejo" $\rightarrow \overline{R}$

Ejemplo: de las palabras a la función

Traducimos cada parte del enunciado a álgebra de Boole:

"es de **noche**" $\rightarrow N$

"**no** lleva ajo" $\rightarrow \overline{A}$

"...*y*..." $\rightarrow$ AND $(\cdot)$

"**no** se refleja en el espejo" $\rightarrow \overline{R}$

"...*o*..." $\rightarrow$

Ejemplo: de las palabras a la función

Traducimos cada parte del enunciado a álgebra de Boole:

"es de **noche**" $\rightarrow N$

"**no** lleva ajo" $\rightarrow \overline{A}$

"...*y*..." $\rightarrow$ AND $(\cdot)$

"**no** se refleja en el espejo" $\rightarrow \overline{R}$

"...*o*..." $\rightarrow$ OR $(+)$

Ejemplo: de las palabras a la función

Traducimos cada parte del enunciado a álgebra de Boole:

"es de **noche**" $\rightarrow N$

"**no** lleva ajo" $\rightarrow \overline{A}$

"...*y*..." $\rightarrow$ AND $(\cdot)$

"**no** se refleja en el espejo" $\rightarrow \overline{R}$

"...*o*..." $\rightarrow$ OR $(+)$

$$V = N \cdot \overline{A} + \overline{R}$$

Ejemplo: tabla de verdad

A partir de la función, construimos la tabla de verdad probando todas las combinaciones de N, A y R:

N	A	R	$V = N \cdot \overline{A} + \overline{R}$
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

Ejemplo: tabla de verdad

A partir de la función, construimos la tabla de verdad probando todas las combinaciones de N, A y R:

N	A	R	$V = N \cdot \overline{A} + \overline{R}$
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

Ni el ajo te salva si no tenés reflejo.

Ejemplo 2

Problema: diseñamos una cerradura digital con dos botones, A y B. La puerta se abre si el usuario presiona primero A y luego, sin soltarlo, presiona B.

Ejemplo 2

Problema: diseñamos una cerradura digital con dos botones, A y B. La puerta se abre si el usuario presiona primero A y luego, sin soltarlo, presiona B.

Identificamos las señales involucradas:

- A: está presionado el botón A
- B: está presionado el botón B
- D: se abre la puerta

¿Alcanzan las entradas actuales?

Si D dependiera sólo de los valores **actuales** de A y B, alcanzaría con una tabla de verdad de 2^2 filas:

A	B	D
0	0	?
0	1	?
1	0	?
1	1	?

¿Alcanzan las entradas actuales?

Si D dependiera sólo de los valores **actuales** de A y B, alcanzaría con una tabla de verdad de 2^2 filas:

A	B	D
0	0	?
0	1	?
1	0	?
1	1	?

Pero el enunciado no depende de **qué** botones están apretados en este instante, sino del **orden** en que se apretaron.

¿Alcanzan las entradas actuales?

Con $A = 1$, $B = 1$ ahora, no podemos saber si D debe valer 1: depende de lo que pasó **antes**.

¿Alcanzan las entradas actuales?

Con $A = 1$, $B = 1$ ahora, no podemos saber si D debe valer 1: depende de lo que pasó **antes**.

Una función booleana sólo puede mirar las entradas en el instante actual: no alcanza para describir un sistema cuya salida depende de su **historia**.

Necesitamos otros conceptos

Para resolver este problema, el sistema necesita **recordar** si ya se presionó A antes de evaluar B: necesita **memoria**.

Necesitamos otros conceptos

Para resolver este problema, el sistema necesita **recordar** si ya se presionó A antes de evaluar B: necesita **memoria**.

Vamos a necesitar nuevas herramientas –**estado** y **circuitos secuenciales**– para describir sistemas cuya salida depende de su historia, no sólo de sus entradas actuales.

Abstracción digital

Señal digital vs. señal analógica

Cualquier magnitud física (tensión, temperatura, presión) puede variar en forma continua: eso es una **señal analógica**.

Señal digital vs. señal analógica

Cualquier magnitud física (tensión, temperatura, presión) puede variar en forma continua: eso es una **señal analógica**.

Señal analógica



Señal digital



Señal digital vs. señal analógica

Cualquier magnitud física (tensión, temperatura, presión) puede variar en forma continua: eso es una **señal analógica**.

Señal analógica



Señal digital

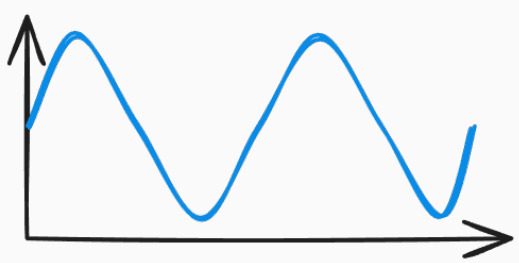


Una señal **digital** sólo puede tomar un número finito de valores válidos (típicamente: **0** y **1**)

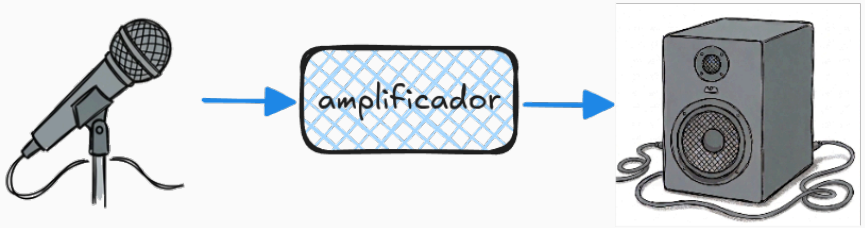
DEPARTAMENTO DE COMPUTACION
Facultad de Ciencias Exactas y Naturales - UBA

Analógico vs Digital

Señal analógica (Continua)



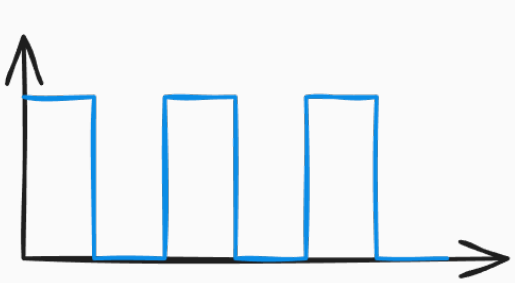
- Señal continua
- Valores infinitos
- Susceptible al ruido



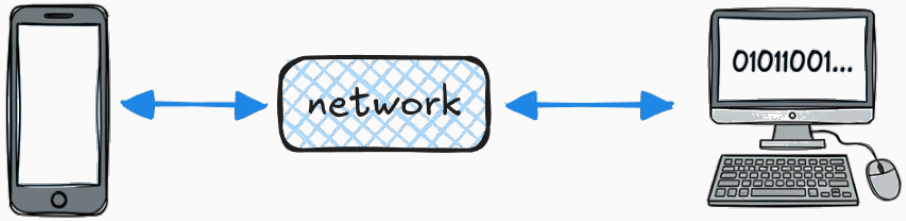
Características:

- Patrón suave de onda
- Representación directa
- Se degrada al copiarla o transmitirla

Señal digital (Discreta)



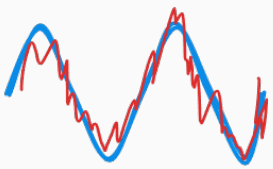
- Señal discreta (0s y 1s)
- Valores finitos
- Mayor inmunidad al ruido



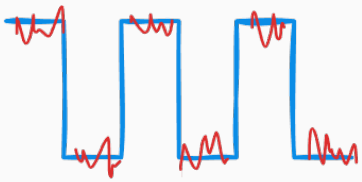
Características:

- Codificada en Binario (Bits)
- Se regenera con facilidad
- sistemas generales (1 bit es 1 bit) y escalables

Inmunidad al ruido



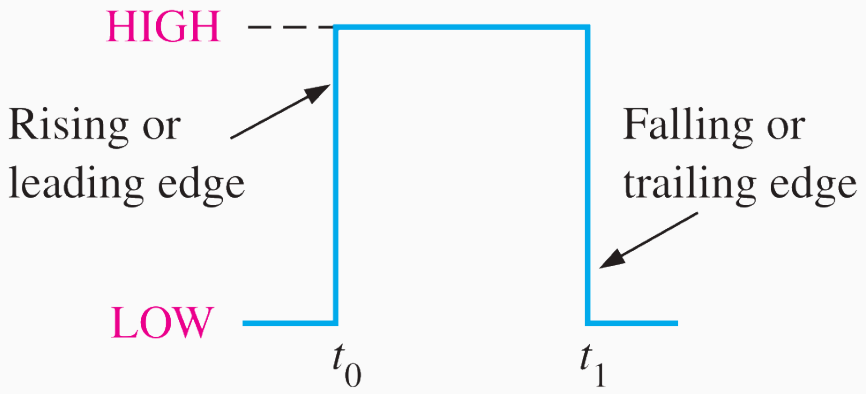
El ruido destruye la señal (Distorsión)

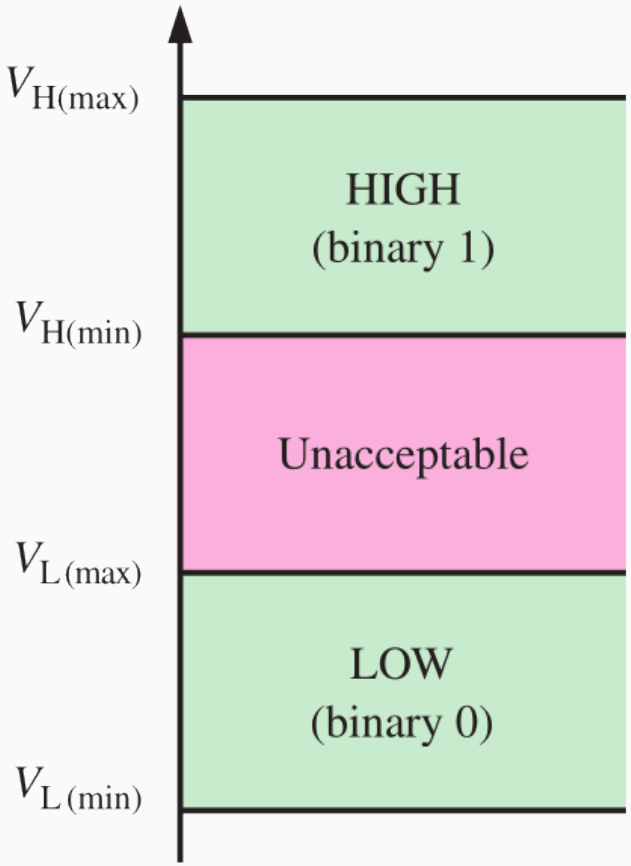


El ruido se elimina de la señal (0s & 1s recuperables)

Abstracción digital

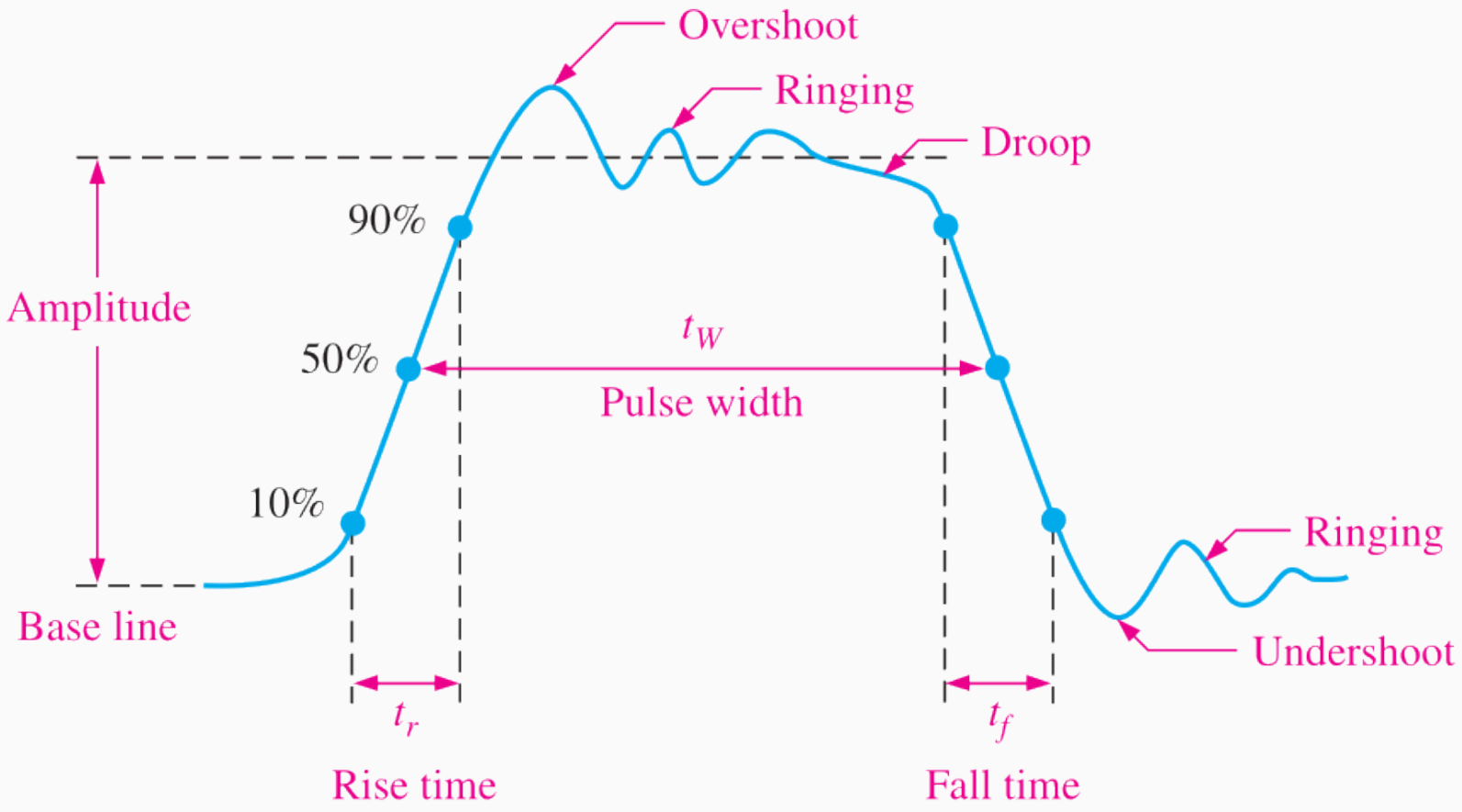
Una tensión se traduce a un símbolo lógico (0 o 1) según el rango de voltajes en el que se encuentre.





Abstracción digital

En la práctica, ninguna tensión es exactamente “0” o “1”: siempre hay ruido, caídas de tensión, variaciones de temperatura...



,

Abstracción digital

La **abstracción digital** consiste en interpretar un rango de voltajes como un mismo símbolo lógico (0 o 1), ignorando las variaciones dentro de ese rango.

Abstracción digital

La **abstracción digital** consiste en interpretar un rango de voltajes como un mismo símbolo lógico (0 o 1), ignorando las variaciones dentro de ese rango.

Esto nos permite:

Abstracción digital

La **abstracción digital** consiste en interpretar un rango de voltajes como un mismo símbolo lógico (0 o 1), ignorando las variaciones dentro de ese rango.

Esto nos permite:

- Diseñar sistemas robustos frente al ruido eléctrico.

Abstracción digital

La **abstracción digital** consiste en interpretar un rango de voltajes como un mismo símbolo lógico (0 o 1), ignorando las variaciones dentro de ese rango.

Esto nos permite:

- Diseñar sistemas robustos frente al ruido eléctrico.
- Razonar con **0** y **1** en lugar de con tensiones concretas.

Abstracción digital

La **abstracción digital** consiste en interpretar un rango de voltajes como un mismo símbolo lógico (0 o 1), ignorando las variaciones dentro de ese rango.

Esto nos permite:

- Diseñar sistemas robustos frente al ruido eléctrico.
- Razonar con **0** y **1** en lugar de con tensiones concretas.
- Construir sistemas arbitrariamente complejos combinando estos símbolos.

Abstracción digital

La **abstracción digital** consiste en interpretar un rango de voltajes como un mismo símbolo lógico (0 o 1), ignorando las variaciones dentro de ese rango.

Esto nos permite:

- Diseñar sistemas robustos frente al ruido eléctrico.
- Razonar con **0** y **1** en lugar de con tensiones concretas.
- Construir sistemas arbitrariamente complejos combinando estos símbolos.

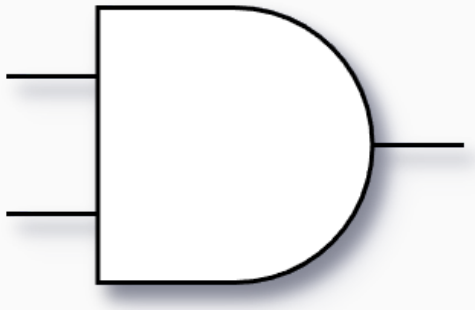
A partir de ahora vamos a trabajar siempre con esta abstracción: **bits**.

Compuertas, tablas de verdad y clasificación de sistemas

Compuertas

Son modelos idealizados de dispositivos electrónicos o de cómputo, que realizan operaciones booleanas.

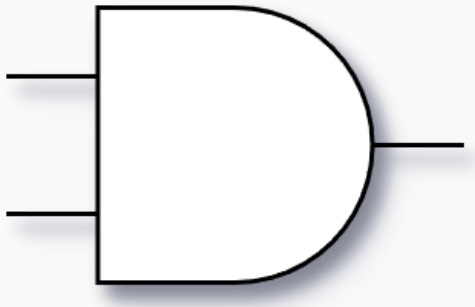
Las podemos representar gráficamente:



Compuertas

Son modelos idealizados de dispositivos electrónicos o de cómputo, que realizan operaciones booleanas.

Las podemos representar gráficamente:



O describir mediante un lenguaje de descripción de hardware (HDL), por ejemplo en `SystemVerilog`:

```sv
assign o = a & b;
```

Tablas de verdad

Son representaciones que nos permiten observar todas las salidas para todas las combinaciones de entradas.

Por ejemplo, la función ($F = X + \overline{Y}$) se representa:

X	Y	F
0	0	
0	1	
1	0	
1	1	

Como resulta esperable, esta representación puede volverse muy compleja cuando el número de variables y salidas crece.

Tablas de verdad

Son representaciones que nos permiten observar todas las salidas para todas las combinaciones de entradas.

Por ejemplo, la función ($F = X + \overline{Y}$) se representa:

X	Y	F
0	0	1
0	1	
1	0	
1	1	

Como resulta esperable, esta representación puede volverse muy compleja cuando el número de variables y salidas crece.

Tablas de verdad

Son representaciones que nos permiten observar todas las salidas para todas las combinaciones de entradas.

Por ejemplo, la función ($F = X + \overline{Y}$) se representa:

X	Y	F
0	0	1
0	1	0
1	0	
1	1	

Como resulta esperable, esta representación puede volverse muy compleja cuando el número de variables y salidas crece.

Tablas de verdad

Son representaciones que nos permiten observar todas las salidas para todas las combinaciones de entradas.

Por ejemplo, la función ($F = X + \overline{Y}$) se representa:

X	Y	F
0	0	1
0	1	0
1	0	1
1	1	

Como resulta esperable, esta representación puede volverse muy compleja cuando el número de variables y salidas crece.

Tablas de verdad

Son representaciones que nos permiten observar todas las salidas para todas las combinaciones de entradas.

Por ejemplo, la función ($F = X + \overline{Y}$) se representa:

X	Y	F
0	0	1
0	1	0
1	0	1
1	1	1

Como resulta esperable, esta representación puede volverse muy compleja cuando el número de variables y salidas crece.

Compuertas - NOT

Gráficamente:

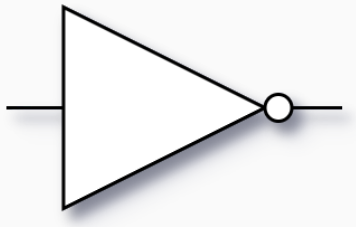


Tabla de verdad:

A	NOT A
0	1
1	0

En SystemVerilog:

```sv
assign o = ~a;
```

Compuertas - AND

Gráficamente:

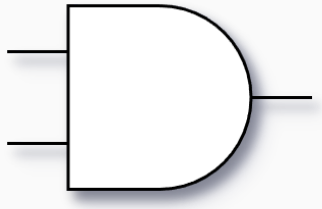


Tabla de verdad:

A	B	A AND B
0	0	0
0	1	0
1	0	0
1	1	1

En SystemVerilog:

```sv
assign o = a & b;
```

Compuertas - OR

Gráficamente:

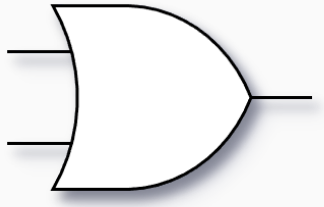


Tabla de verdad:

A	B	A OR B
0	0	0
0	1	1
1	0	1
1	1	1

En SystemVerilog:

```sv
assign o = a | b;
```

Compuertas - XOR u OR-EXCLUSIVA

Gráficamente:

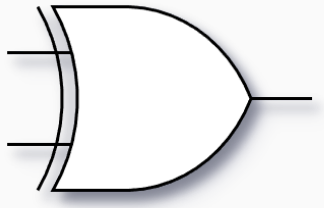


Tabla de verdad:

A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

En SystemVerilog:

```sv
assign o = a ^ b;
```

Compuertas - NAND

Gráficamente:

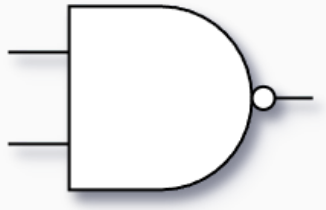


Tabla de verdad:

A	B	A NAND B
0	0	1
0	1	1
1	0	1
1	1	0

En SystemVerilog:

```sv
assign o = ~(a & b);
```

Compuertas - NOR

Gráficamente:

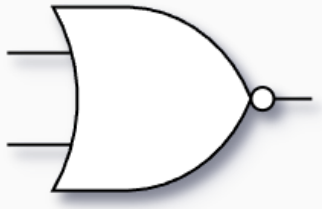


Tabla de verdad:

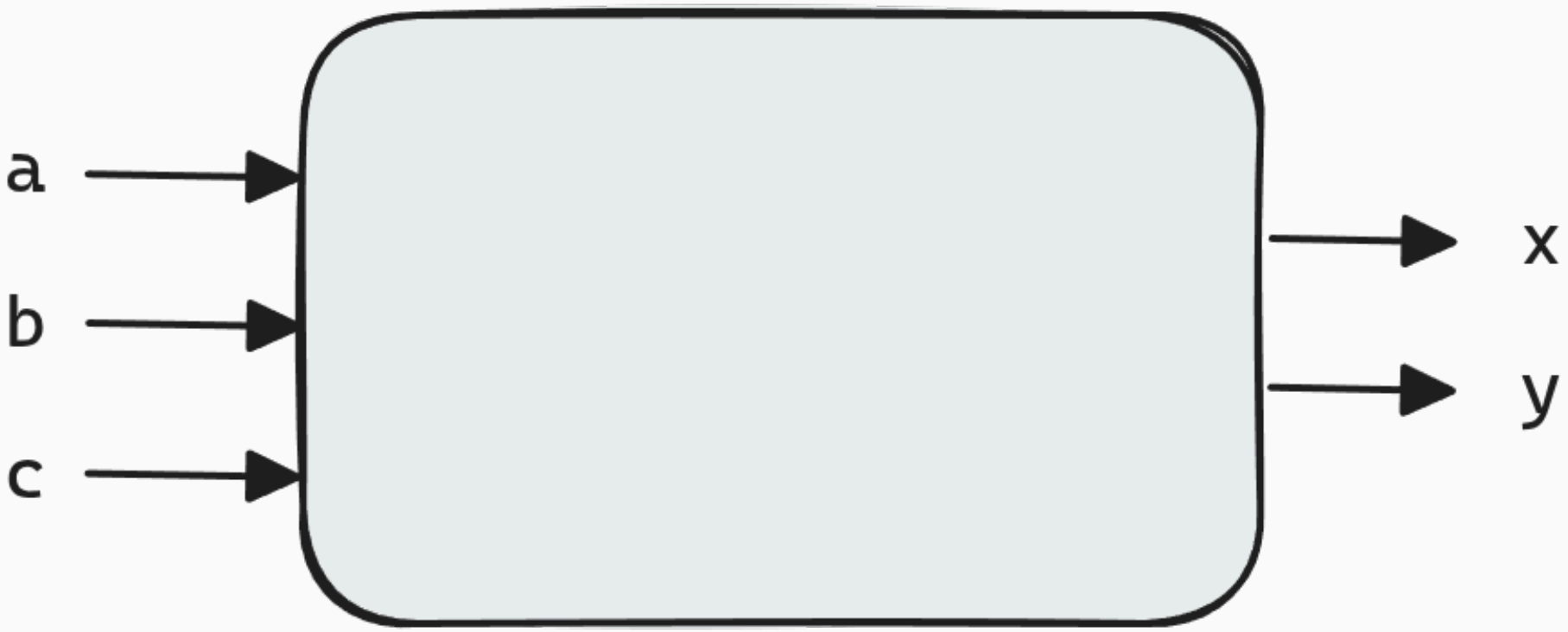
A	B	A NOR B
0	0	1
0	1	0
1	0	0
1	1	0

En SystemVerilog:

```sv
assign o = ~(a | b);
```

Un sistema digital es una red que procesa variables con valores discretos.

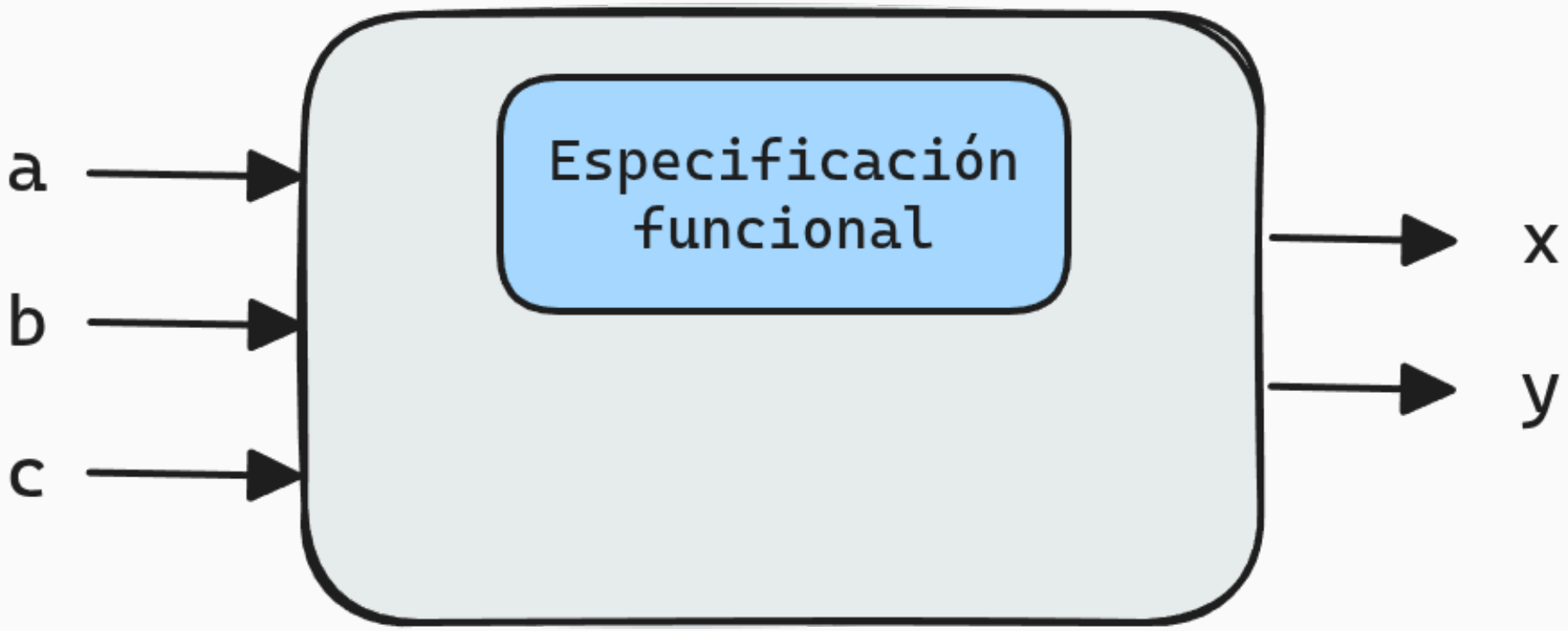
Un sistema puede ser visto como una *caja negra*:



Sistema digital

Un sistema digital es una red que procesa variables con valores discretos.

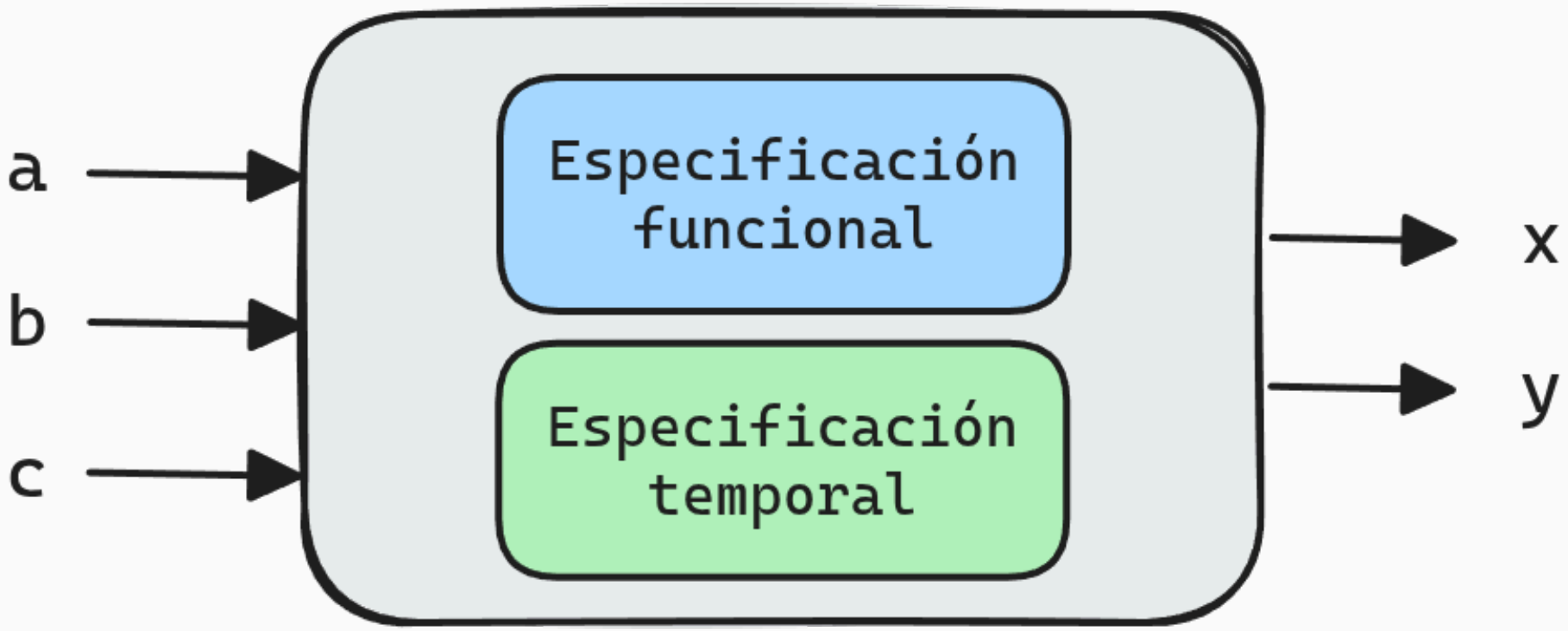
Un sistema puede ser visto como una *caja negra*:



Sistema digital

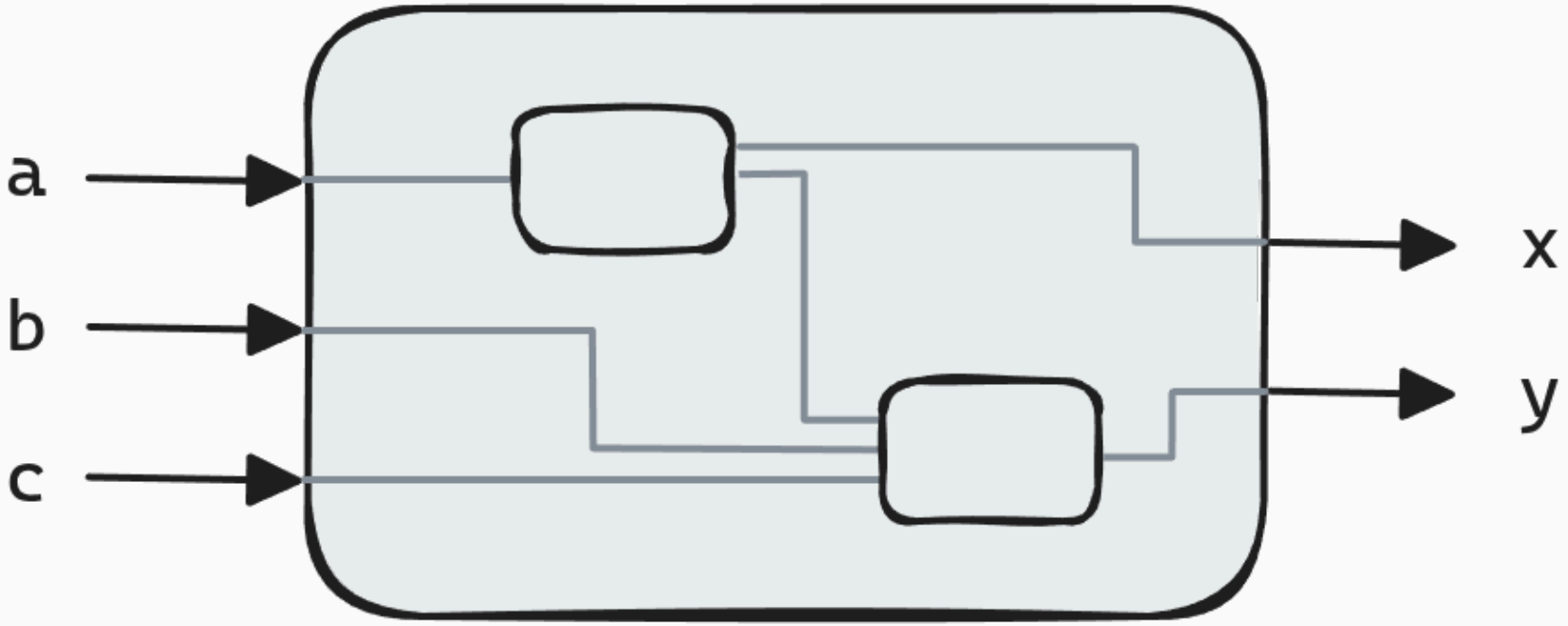
Un sistema digital es una red que procesa variables con valores discretos.

Un sistema puede ser visto como una *caja negra*:



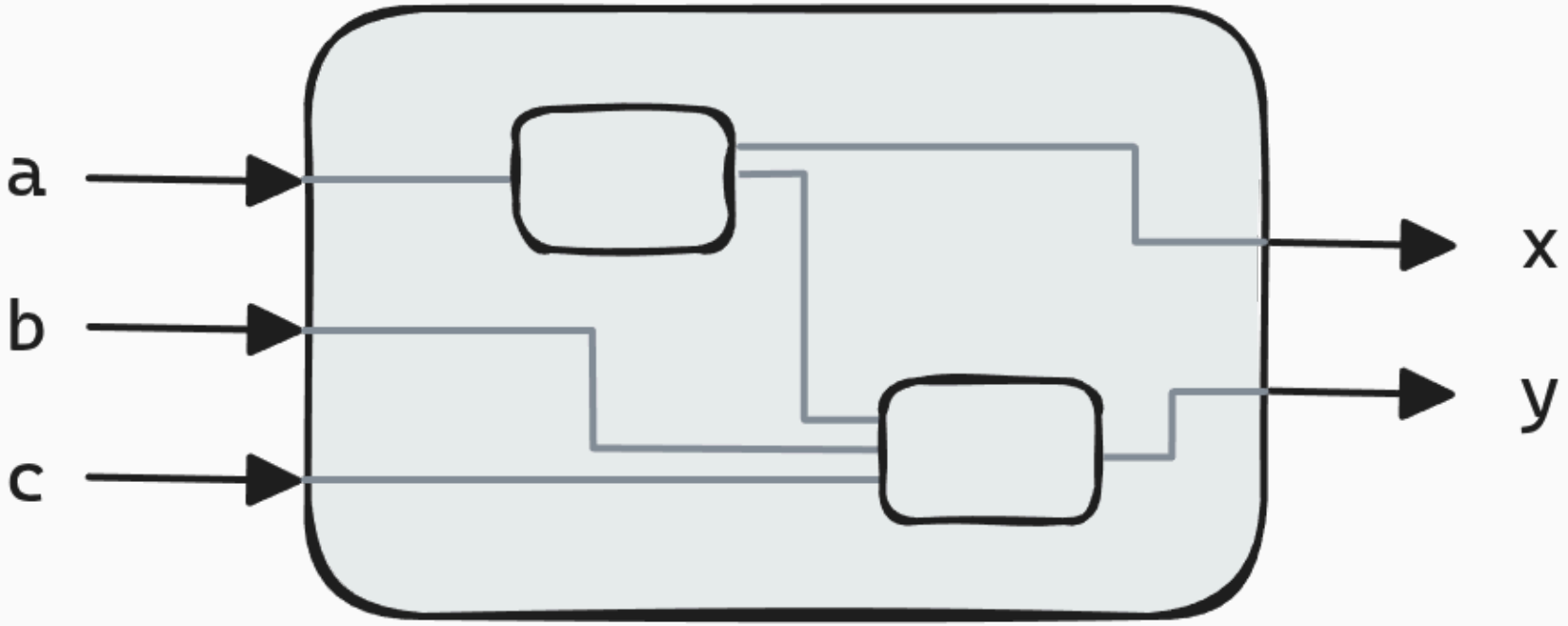
Sistema digital

A su vez, un sistema puede estar compuesto por subsistemas, que a su vez pueden estar compuestos por otros subsistemas, y así sucesivamente.



Sistema digital

A su vez, un sistema puede estar compuesto por subsistemas, que a su vez pueden estar compuestos por otros subsistemas, y así sucesivamente.



Generalmente, usamos el nivel de **abstracción** que más nos conviene según el problema que estemos resolviendo.

Sistema combinacional

Un sistema digital es **combinacional** si:

- las salidas dependen únicamente de las entradas actuales;
- no tiene memoria ni estado interno;
- para cada combinación de entradas, la salida queda completamente determinada.

Sistemas combinacionales

Sistema combinacional

Un sistema digital es **combinacional** si:

- las salidas dependen únicamente de las entradas actuales;
- no tiene memoria ni estado interno;
- para cada combinación de entradas, la salida queda completamente determinada.

En esta clase (y la siguiente) vamos a trabajar exclusivamente con sistemas combinacionales.

Ejemplos y contraejemplos

La luz de un auto se enciende si el motor está apagado y una puerta está abierta.

Ejemplos y contraejemplos

La luz de un auto se enciende si el motor está apagado y una puerta está abierta. → **Combinacional**

Ejemplos y contraejemplos

La luz de un auto se enciende si el motor está apagado y una puerta está abierta. → **Combinacional**

Un semáforo típico para automóviles.

Ejemplos y contraejemplos

La luz de un auto se enciende si el motor está apagado y una puerta está abierta. → **Combinacional**

Un semáforo típico para automóviles. → **No combinacional**

Ejemplos y contraejemplos

La luz de un auto se enciende si el motor está apagado y una puerta está abierta. → **Combinacional**

Un semáforo típico para automóviles. → **No combinacional**

Un contador que me indica cuál es la siguiente línea de un programa que debo ejecutar

Ejemplos y contraejemplos

La luz de un auto se enciende si el motor está apagado y una puerta está abierta. → **Combinacional**

Un semáforo típico para automóviles. → **No combinacional**

Un contador que me indica cuál es la siguiente línea de un programa que debo ejecutar → **No combinacional**

Ejemplos y contraejemplos

La luz de un auto se enciende si el motor está apagado y una puerta está abierta. → **Combinacional**

Un semáforo típico para automóviles. → **No combinacional**

Un contador que me indica cuál es la siguiente línea de un programa que debo ejecutar → **No combinacional**

Un sistema de 2 teclas para una luz de escalera

Ejemplos y contraejemplos

La luz de un auto se enciende si el motor está apagado y una puerta está abierta. → **Combinacional**

Un semáforo típico para automóviles. → **No combinacional**

Un contador que me indica cuál es la siguiente línea de un programa que debo ejecutar → **No combinacional**

Un sistema de 2 teclas para una luz de escalera → **Combinacional**

Ejemplos y contraejemplos

La luz de un auto se enciende si el motor está apagado y una puerta está abierta. → **Combinacional**

Un semáforo típico para automóviles. → **No combinacional**

Un contador que me indica cuál es la siguiente línea de un programa que debo ejecutar → **No combinacional**

Un sistema de 2 teclas para una luz de escalera → **Combinacional**

Si la salida depende de la **historia** del sistema, no alcanza con las entradas actuales: hace falta memoria.

Representación de la información - Introducción: enteros sin signo

¿Cómo representamos números naturales?

Números decimales

Dígitos: $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$

$$3861_{10}$$

¿Cómo representamos números naturales?

Números decimales

Dígitos: $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$

$$3861_{10} = 3 \cdot 10^3 + 8 \cdot 10^2 + 6 \cdot 10^1 + 1 \cdot 10^0$$

¿Cómo representamos números naturales?

Números decimales

Dígitos: $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$

$$3861_{10} = 3 \cdot 10^3 + 8 \cdot 10^2 + 6 \cdot 10^1 + 1 \cdot 10^0$$

Un número natural **decimal** de N dígitos representa una de las 10^N combinaciones posibles:

$$0, 1, ..., 10^N - 1$$

Las potencias de 10 son: $1, 10, 100, 1000, 10000...$

¿Cómo representamos números naturales?

Números decimales

Dígitos: $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$

$$3861_{10} = 3 \cdot 10^3 + 8 \cdot 10^2 + 6 \cdot 10^1 + 1 \cdot 10^0$$

Un número natural **decimal** de N dígitos representa una de las 10^N combinaciones posibles:

$$0, 1, ..., 10^N - 1$$

Las potencias de 10 son: $1, 10, 100, 1000, 10000...$

Nos referiremos a la representación de números tradicional como **números en base 10**

¿Cómo representamos números naturales?

Números binarios

Dígitos: $\{0, 1\}$

$$1011_2$$

¿Cómo representamos números naturales?

Números binarios

Dígitos: $\{0, 1\}$

$$1011_2 = 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0$$

¿Cómo representamos números naturales?

Números binarios

Dígitos: $\{0, 1\}$

$$1011_2 = 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0$$

Un número natural **binario** de N dígitos representa una de las 2^N combinaciones posibles:

$$0, 1, ..., 2^N - 1$$

. Las potencias de 2 son: $1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024...$

¿Cómo representamos números naturales?

Números binarios

Dígitos: $\{0, 1\}$

$$1011_2 = 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0$$

Un número natural **binario** de N dígitos representa una de las 2^N combinaciones posibles:

$$0, 1, ..., 2^N - 1$$

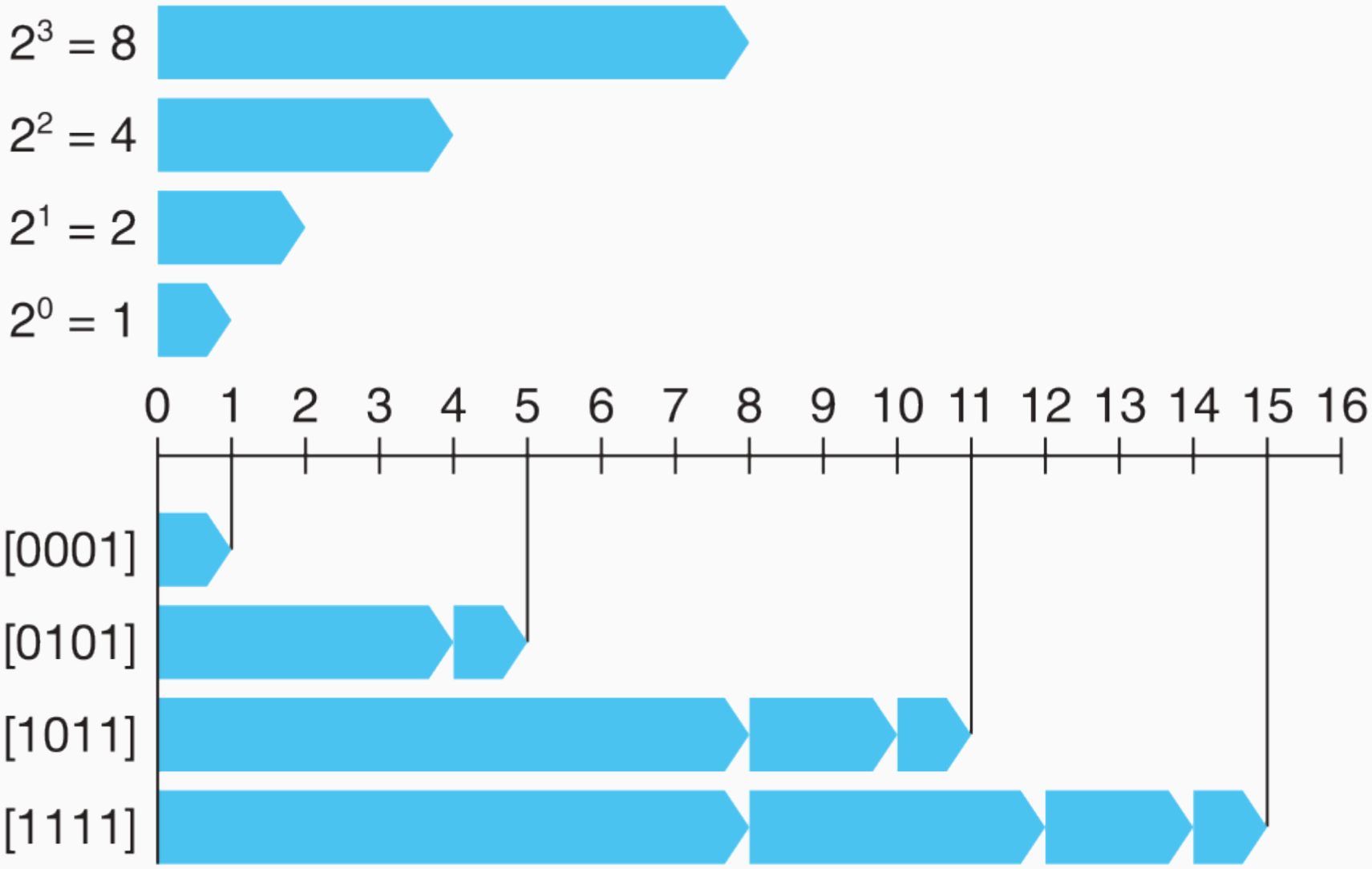
. Las potencias de 2 son: $1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024...$

Nos referiremos a la representación de números binarios como **números en base 2**

Equivalencias

1-bit	2-bit	3-bit	4-bit	Decimal	Pesos
0	00	000	0000	0	$0 \cdot 2^0$
1	01	001	0001	1	$1 \cdot 2^0$
	10	010	0010	2	$1 \cdot 2^1 + 0 \cdot 2^0$
	11	011	0011	3	$1 \cdot 2^1 + 1 \cdot 2^0$
		100	0100	4	$1 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0$
		101	0101	5	$1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0$
		110	0110	6	$1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0$
		111	0111	7	$1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0$
			1000	8	$1 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0$
			1001	9	$1 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0$
			1010	10	$1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0$
			1011	11	$1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0$
			1100	12	$1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0$
			1101	13	$1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0$
			1110	14	$1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0$
			1111	15	$1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0$

Números naturales sin signo



Fuente: Bryant & Hallaron, Computer Systems, 2019, capítulo 2, 2.2

Sistema Binario

En base 2, sólo tenemos dos dígitos: $\{0, 1\}$.

Decimal	Binario	Descomposición
5	101	$1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0$

Sistema Binario

En base 2, sólo tenemos dos dígitos: $\{0, 1\}$.

Decimal	Binario	Descomposición
5	101	$1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0$
17	10001	$1 \cdot 2^4 + 0 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0$

Sistema Binario

En base 2, sólo tenemos dos dígitos: $\{0, 1\}$.

Decimal	Binario	Descomposición
5	101	$1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0$
17	10001	$1 \cdot 2^4 + 0 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0$
23	10111	$1 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0$

Sistema Binario

En base 2, sólo tenemos dos dígitos: $\{0, 1\}$.

Decimal	Binario	Descomposición
5	101	$1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0$
17	10001	$1 \cdot 2^4 + 0 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0$
23	10111	$1 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0$
42	101010	$1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0$

Sistema Binario

En base 2, sólo tenemos dos dígitos: $\{0, 1\}$.

Decimal	Binario	Descomposición
5	101	$1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0$
17	10001	$1 \cdot 2^4 + 0 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0$
23	10111	$1 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0$
42	101010	$1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0$
100	1100100	$1 \cdot 2^6 + 1 \cdot 2^5 + 0 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0$

Sistema Binario

En base 2, sólo tenemos dos dígitos: $\{0, 1\}$.

Podemos deshacernos de los ceros en la descomposición:

Decimal	Binario	Descomposición
5	101	$1 \cdot 2^2 + 1 \cdot 2^0$
17	10001	$1 \cdot 2^4 + 1 \cdot 2^0$
23	10111	$1 \cdot 2^4 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0$
42	101010	$1 \cdot 2^5 + 1 \cdot 2^3 + 1 \cdot 2^1$
100	1100100	$1 \cdot 2^6 + 1 \cdot 2^5 + 1 \cdot 2^2$

Cambio de bases

Teorema de la división

Si $a, b \in \mathbb{Z}$ con $b > 0$, existen enteros únicos q, r tal que

$$a = qb + r, 0 \leq r < b$$

En este teorema, se debería pensar a a como el *dividendo*, b como el *divisor*, q como el cociente y r como el resto

Cambio de bases

Teorema de la división

Si $a, b \in \mathbb{Z}$ con $b > 0$, existen enteros únicos q, r tal que

$$a = qb + r, 0 \leq r < b$$

En este teorema, se debería pensar a a como el *dividendo*, b como el *divisor*, q como el cociente y r como el resto

¿Cómo lo usamos?

$a = q \cdot b + r_0$

Cambio de bases

Teorema de la división

Si $a, b \in \mathbb{Z}$ con $b > 0$, existen enteros únicos q, r tal que

$$a = qb + r, 0 \leq r < b$$

En este teorema, se debería pensar a a como el *dividendo*, b como el *divisor*, q como el cociente y r como el resto

¿Cómo lo usamos?

$a = q \cdot b + r_0$
$a = (q_1 \cdot b + r_1) \cdot b + r_0$

Cambio de bases

Teorema de la división

Si $a, b \in \mathbb{Z}$ con $b > 0$, existen enteros únicos q, r tal que

$$a = qb + r, 0 \leq r < b$$

En este teorema, se debería pensar a a como el *dividendo*, b como el *divisor*, q como el cociente y r como el resto

¿Cómo lo usamos?

$$a = q \cdot b + r_0$$
$$a = (q_1 \cdot b + r_1) \cdot b + r_0$$
$$a = [(q_2 \cdot b + r_2) \cdot b + r_1] \cdot b + r_0$$

Cambio de bases

Teorema de la división

Si $a, b \in \mathbb{Z}$ con $b > 0$, existen enteros únicos q, r tal que

$$a = qb + r, 0 \leq r < b$$

En este teorema, se debería pensar a a como el *dividendo*, b como el *divisor*, q como el cociente y r como el resto

¿Cómo lo usamos?

$$a = q \cdot b + r_0$$
$$a = (q_1 \cdot b + r_1) \cdot b + r_0$$
$$a = [(q_2 \cdot b + r_2) \cdot b + r_1] \cdot b + r_0$$

Podemos continuar con la expansión hasta que $q_{N-1} < b$

$$a = \{[(q_{N-1} \cdot b + r_{N-1}) \cdot b + r_{N-2}] \cdot b + \cdots\} \cdot b + r_0 \cdot 1$$

Cambio de bases

Teorema de la división

Si $a, b \in \mathbb{Z}$ con $b > 0$, existen enteros únicos q, r tal que

$$a = qb + r, 0 \leq r < b$$

En este teorema, se debería pensar a a como el *dividendo*, b como el *divisor*, q como el cociente y r como el resto

¿Cómo lo usamos?

$a = q \cdot b + r_0$
$a = (q_1 \cdot b + r_1) \cdot b + r_0$
$a = [(q_2 \cdot b + r_2) \cdot b + r_1] \cdot b + r_0$

Podemos continuar con la expansión hasta que $q_{N-1} < b$

$$a = \{[(q_{N-1} \cdot b + r_{N-1}) \cdot b + r_{N-2}] \cdot b + \cdots\} \cdot b + r_0 \cdot 1$$

Si distribuimos va a quedar:

$$a = q_{N-1} \cdot b^N + r_{N-1} \cdot b^{N-1} + \cdots + r_1 \cdot b + r_0 \cdot b^0$$

Representación posicional:

Podemos ver que el primer elemento de cada producto es el que va a aparecer en los dígitos de nuestra representación posicional, donde:

$$a = q_{N-1} \cdot b^N + r_{N-1} \cdot b^{N-1} + \cdots + r_1 \cdot b + r_0 \cdot b^0$$

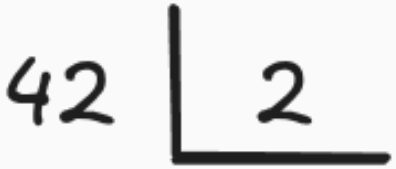
Se puede escribir como:

q_{N-1}	r_{N-1}	$\cdots$	r_1	r_0

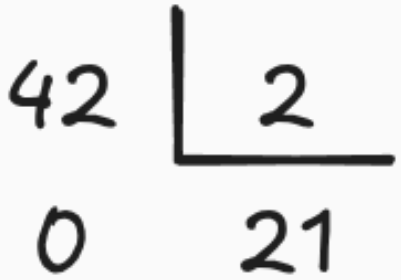
O de forma correcta, incluyendo la base

$$(q_{N-1} r_{N-1} \cdots r_1 r_0)_b$$

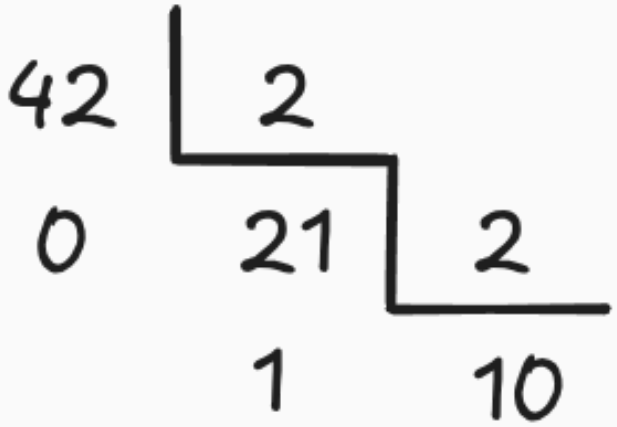
Cambio de bases



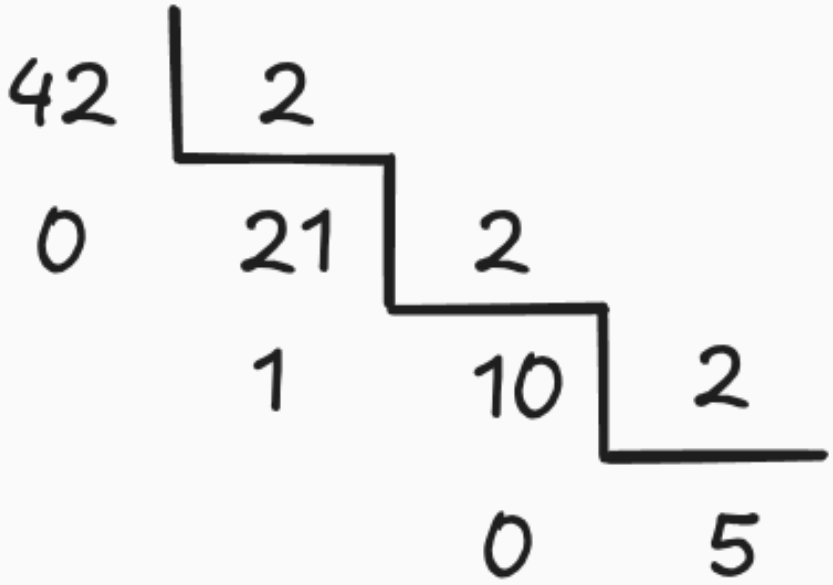
Cambio de bases



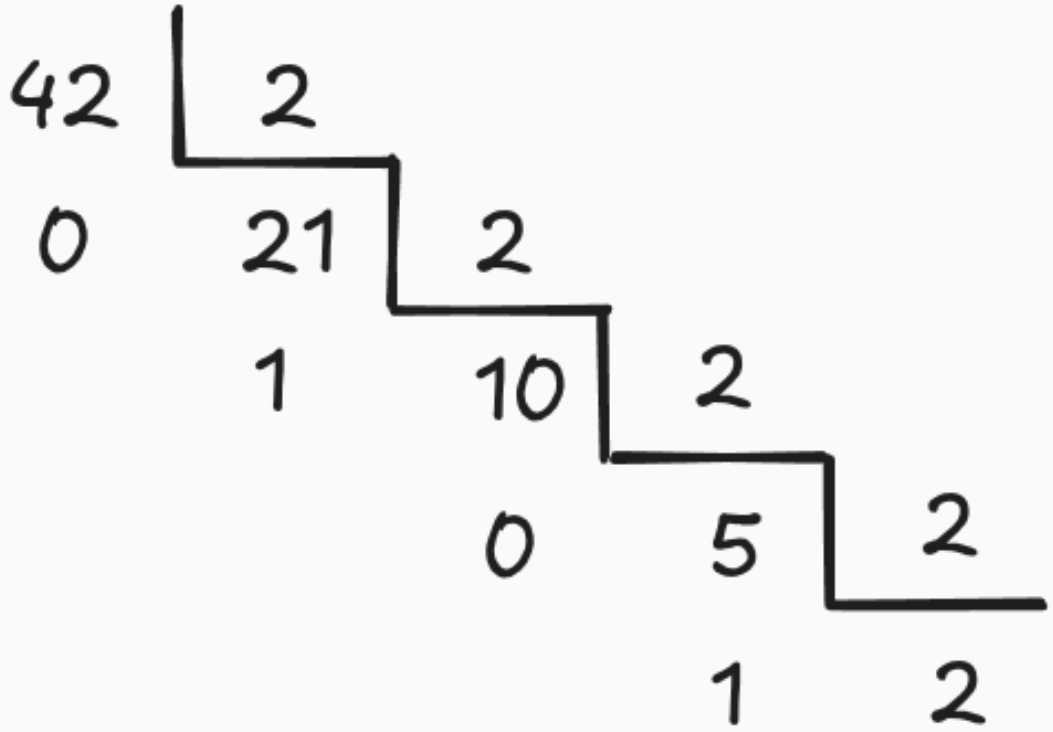
Cambio de bases



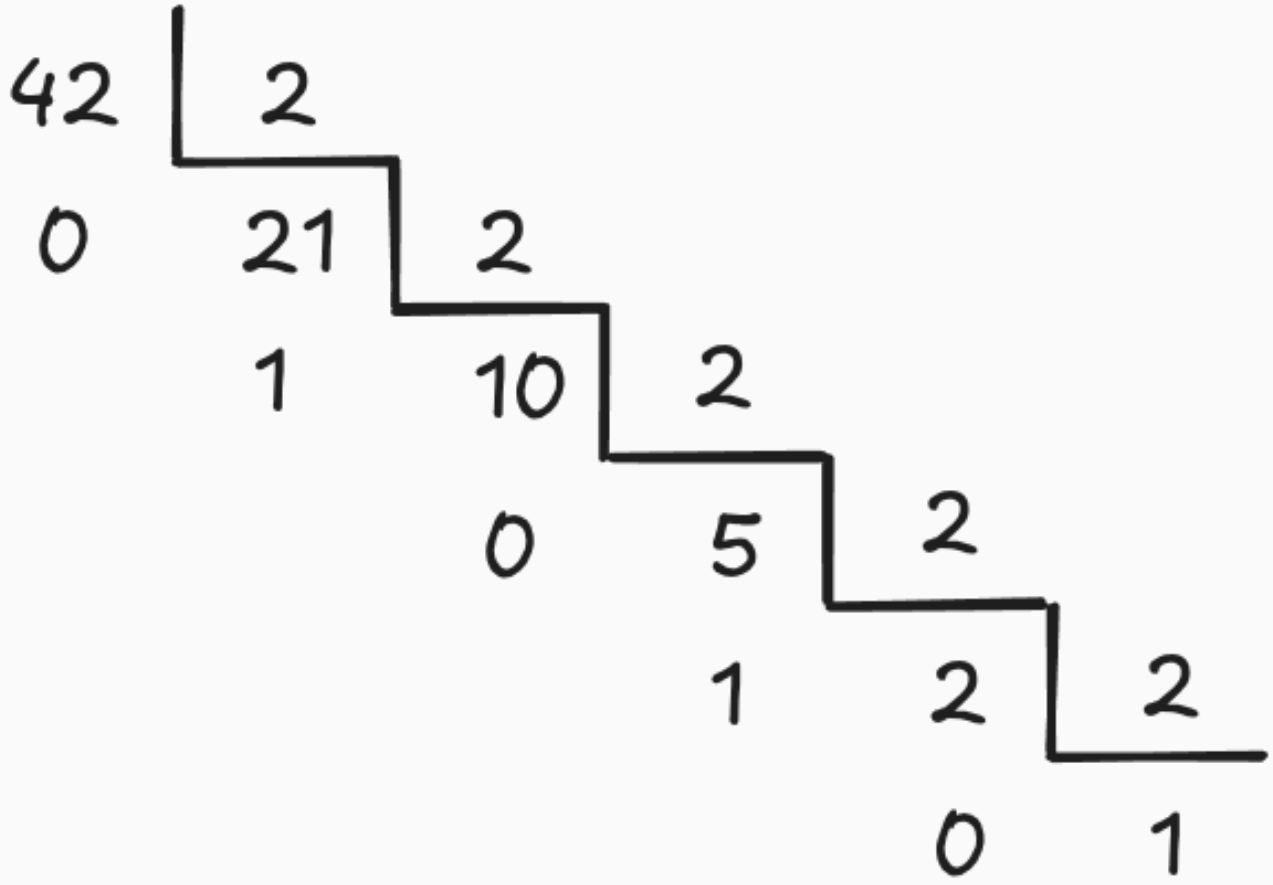
Cambio de bases



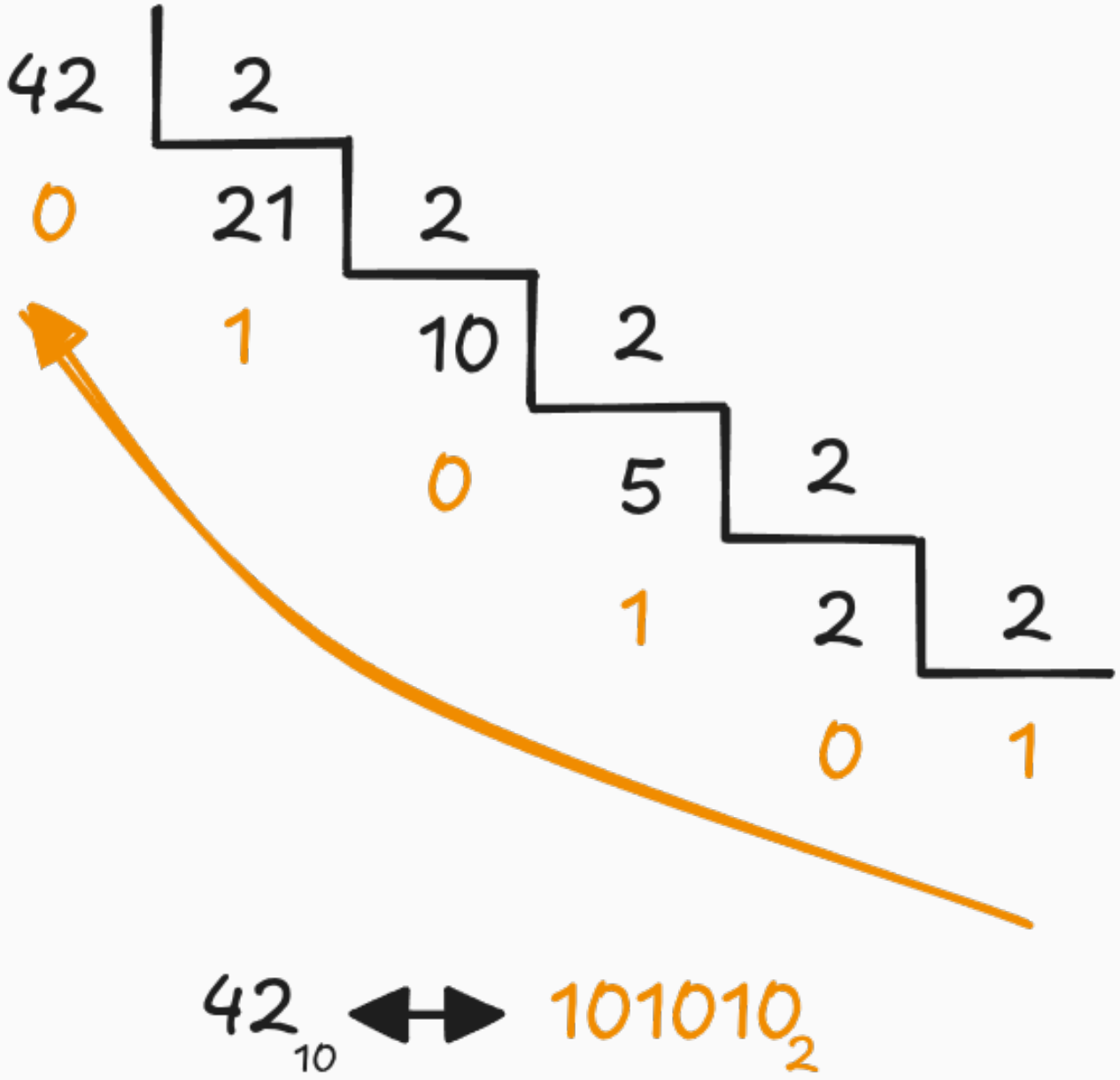
Cambio de bases



Cambio de bases

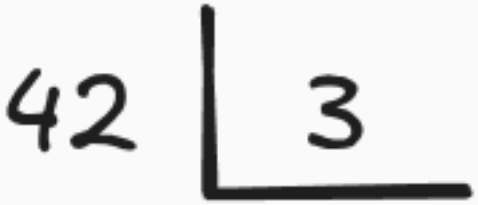


Cambio de bases



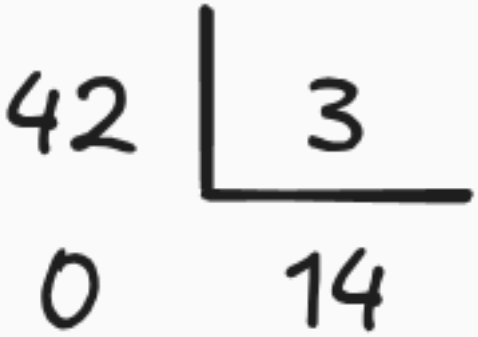
Cambio de bases

Funciona en cualquier base:



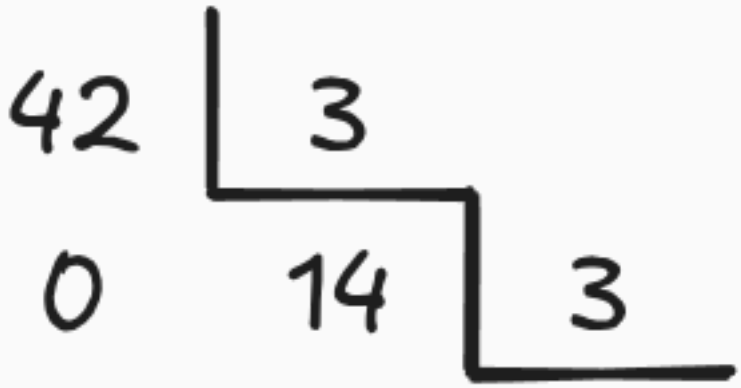
Cambio de bases

Funciona en cualquier base:



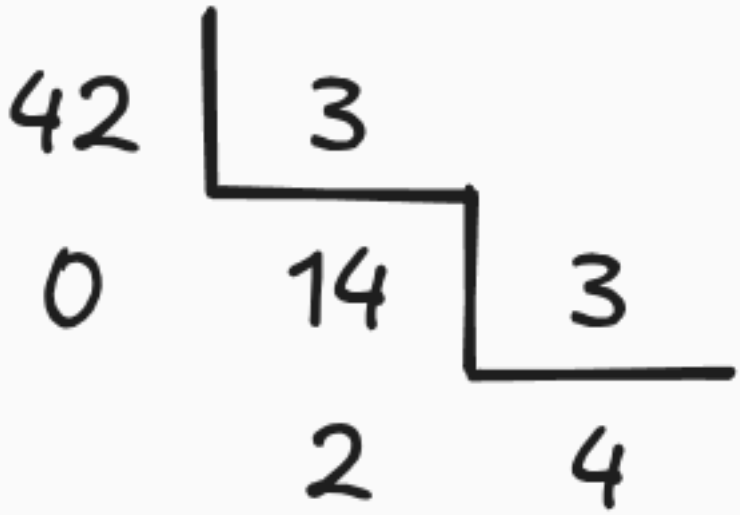
Cambio de bases

Funciona en cualquier base:



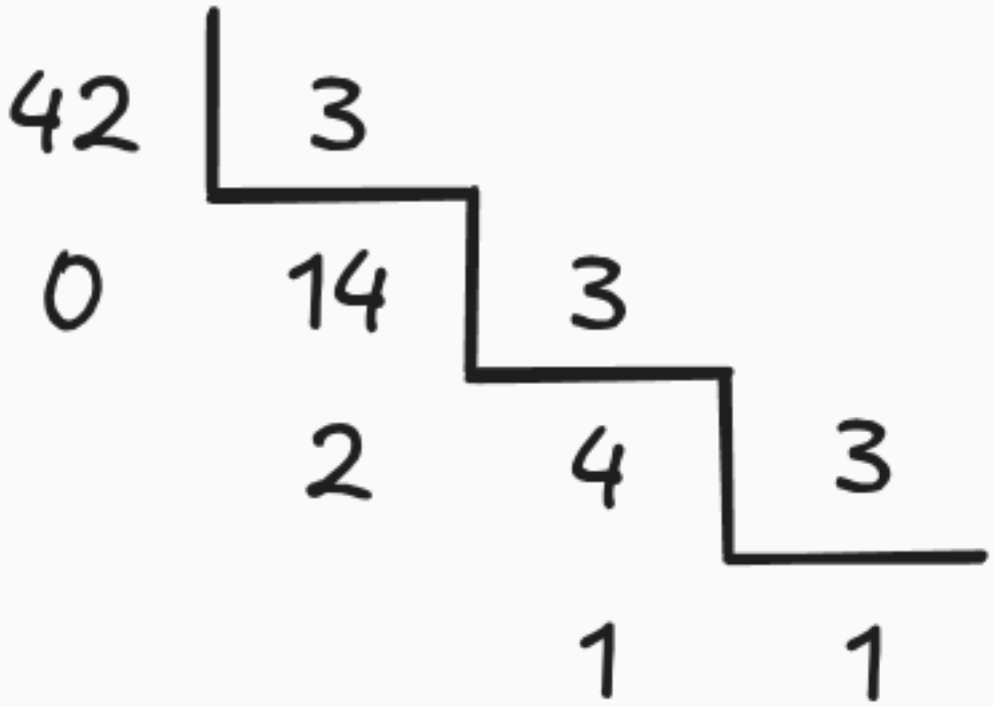
Cambio de bases

Funciona en cualquier base:



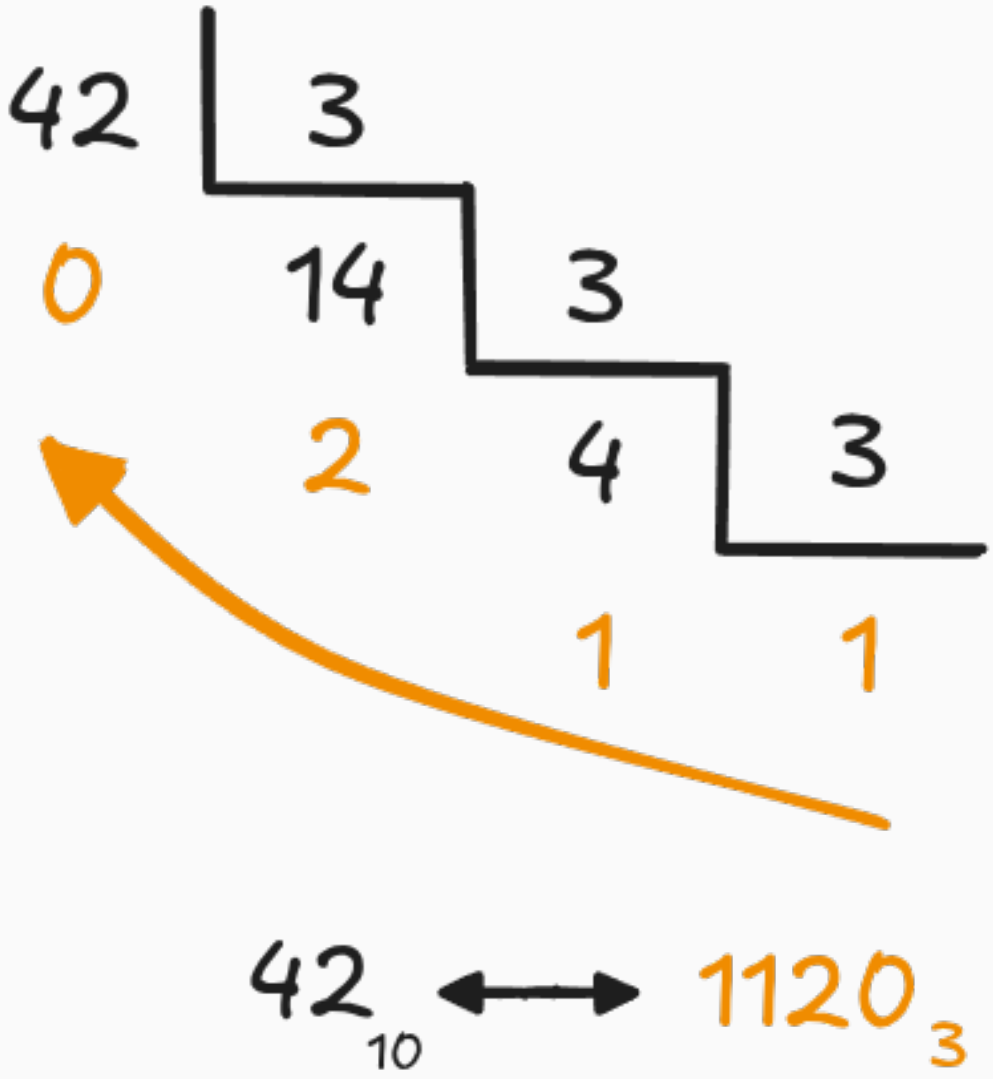
Cambio de bases

Funciona en cualquier base:



Cambio de bases

Funciona en cualquier base:



Representación hexadecimal

Para mayor comodidad, a menudo se agrupan los dígitos binarios de a 4, lo que da lugar a la representación **hexadecimal**:

$$66532_{10} = 1000000111100100_2$$

Agrupo de a **4 dígitos**:

$$0001\ 0000\ 0011\ 1110\ 0100$$

Convierto cada **nibble** a su equivalente hexadecimal:

$$0x103E4$$

Lo que puede ser verificado con la descomposición:

$$1 \cdot 16^4 + 0 \cdot 16^3 + 3 \cdot 16^2 + \mathrm{E} \cdot 16^1 + 4 \cdot 16^0$$

Dec	Hexa	Bin
0	0x0	0000
1	0x1	0001
2	0x2	0010
3	0x3	0011
4	0x4	0100
5	0x5	0101
6	0x6	0110
7	0x7	0111
8	0x8	1000
9	0x9	1001
10	0xA	1010
11	0xB	1011
12	0xC	1100
13	0xD	1101
14	0xE	1110
15	0xF	1111

Un poco de nomenclatura

- 1 **byte** = 8 bits

Un poco de nomenclatura

- 1 **byte** = 8 bits
  - Binario: $00000000_2 \rightarrow 11111111_2$

Un poco de nomenclatura

- 1 **byte** = 8 bits
  - Binario: $00000000_2 \rightarrow 11111111_2$
  - Hexadecimal: $00_{16} \rightarrow \mathrm{FF}_{16}$

Un poco de nomenclatura

- 1 **byte** = 8 bits
  - Binario: $00000000_2 \rightarrow 11111111_2$
  - Hexadecimal: $00_{16} \rightarrow \mathrm{FF}_{16}$
  - Decimal: $0_{10} \rightarrow 255_{10}$

Un poco de nomenclatura

- 1 **byte** = 8 bits
  - Binario: $00000000_2 \rightarrow 11111111_2$
  - Hexadecimal: $00_{16} \rightarrow \mathrm{FF}_{16}$
  - Decimal: $0_{10} \rightarrow 255_{10}$
- 1 **nibble** = 4 bits

Un poco de nomenclatura

- 1 **byte** = 8 bits
  - Binario: $00000000_2 \rightarrow 11111111_2$
  - Hexadecimal: $00_{16} \rightarrow \mathrm{FF}_{16}$
  - Decimal: $0_{10} \rightarrow 255_{10}$
- 1 **nibble** = 4 bits

- 1 kilobyte (KB) = 1024 bytes = 2^{10} bytes

Un poco de nomenclatura

- 1 **byte** = 8 bits
  - Binario: $00000000_2 \rightarrow 11111111_2$
  - Hexadecimal: $00_{16} \rightarrow \mathrm{FF}_{16}$
  - Decimal: $0_{10} \rightarrow 255_{10}$
- 1 **nibble** = 4 bits

- 1 kilobyte (KB) = 1024 bytes = 2^{10} bytes
- 1 megabyte (MB) = 1024 KB = 2^{20} bytes

Un poco de nomenclatura

- 1 **byte** = 8 bits
  - Binario: $00000000_2 \rightarrow 11111111_2$
  - Hexadecimal: $00_{16} \rightarrow \mathrm{FF}_{16}$
  - Decimal: $0_{10} \rightarrow 255_{10}$
- 1 **nibble** = 4 bits

- 1 kilobyte (KB) = 1024 bytes = 2^{10} bytes
- 1 megabyte (MB) = 1024 KB = 2^{20} bytes
- 1 gigabyte (GB) = 1024 MB = 2^{30} bytes

Un poco de nomenclatura

- 1 **byte** = 8 bits
  - Binario: $00000000_2 \rightarrow 11111111_2$
  - Hexadecimal: $00_{16} \rightarrow \mathrm{FF}_{16}$
  - Decimal: $0_{10} \rightarrow 255_{10}$
- 1 **nibble** = 4 bits

- 1 kilobyte (KB) = 1024 bytes = 2^{10} bytes
- 1 megabyte (MB) = 1024 KB = 2^{20} bytes
- 1 gigabyte (GB) = 1024 MB = 2^{30} bytes
- 1 terabyte (TB) = 1024 GB = 2^{40} bytes

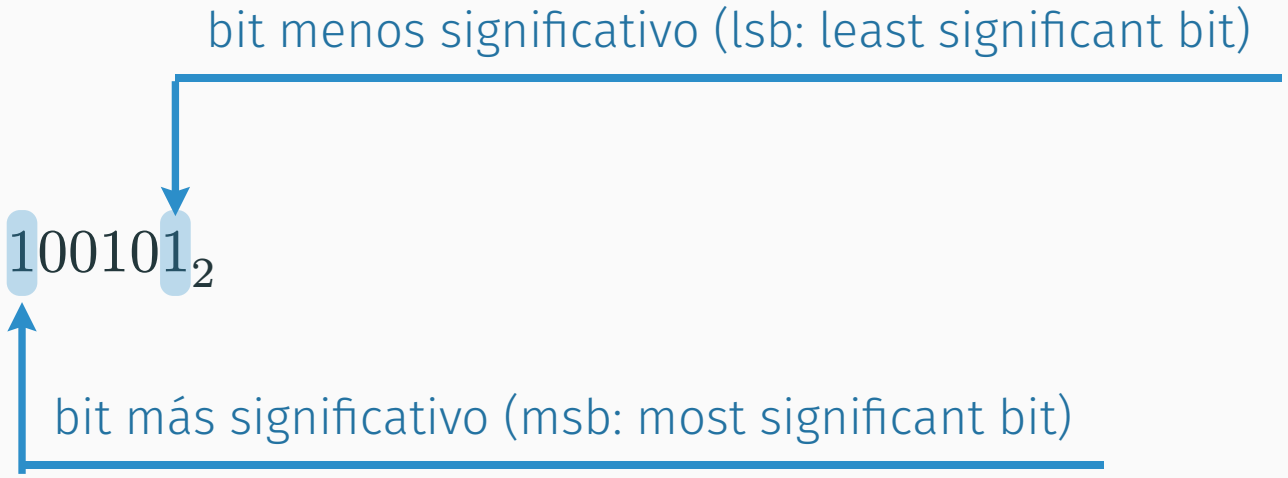
En realidad 1024 bytes es un kibibyte (KiB), 1024 KiB es un mebibyte (MiB), etc. Pero el uso común es usar KB, MB, etc. para referirse a potencias de 1024. Ver https://en.wikipedia.org/wiki/Byte#Multiple-byte_units para más detalles.

Un poco de nomenclatura

100101_2

bit más significativo (msb: most significant bit)

Un poco de nomenclatura



Un poco de nomenclatura

bit menos significativo (lsb: least significant bit)

100101_2

bit más significativo (msb: most significant bit)

$DEADBEEF_{16}$

byte más significativo (MSB: most significant byte)

Un poco de nomenclatura

